THINKHIVELABS

Nexiqon by ThinkHiveLabs

Product Strategy, Technical Architecture, SaaS Security & Implementation Specification

Document status: Baseline technical design for product planning (not an implemented or certified system).

Version: 1.1 Date: 07 November 2026 Owner: ThinkHiveLabs

Positioning: Secure, Configurable Enterprise Agentic AI Platform.

PRODUCT PRINCIPLE	COMMERCIAL PRINCIPLE
Build one core with reusable RAG, agents, security and plugins.	Sell initial implementations; scale into managed SaaS and dedicated/private licensing.

Confidential - internal product planning. All dates, prices and targets are proposals requiring validation.

1. Executive decision

Nexiqon by ThinkHiveLabs will be developed as a reusable, configuration-driven AI agent product. It will initially be commercialized through paid implementation engagements and subsequently offered as shared SaaS, dedicated SaaS and private deployment. A common codebase will serve all deployment models; integrations, data connectors, policies and agent behaviors will be configurable.

Scope boundary: The platform is not merely an LLM API wrapper. It supplies secure retrieval, source citations, stateful workflows, controlled tools, tenant isolation, evaluation, governance and operational interfaces. A production claim must be based on implemented and tested controls, not this design document.

Target opportunities and product templates

Template	Typical knowledge	Tools and workflows
Customer Support	FAQs, support articles, policies	Ticket creation, troubleshooting, escalation
SaaS Knowledge Assistant	Product docs, uploaded files	Account/subscription APIs, support workflows
E-commerce	Catalog, policies	Product recommendations, orders, returns
Industrial Service	Manuals, service documentation	Diagnostics, maintenance workflows, service tickets
Enterprise Knowledge	SOPs, internal knowledge	ACL-aware retrieval and human handoff

Non-goals for the MVP

- General-purpose autonomous agents with unrestricted execution privileges.
- A fully featured visual drag-and-drop workflow builder or dozens of first-party connectors.
- Training or fine-tuning a foundational model as a prerequisite.
- Claims of zero hallucinations, zero breach risk, or compliance certification before independent validation.

2. Functional requirements

ID	Requirement	MVP acceptance direction
FR-01	Agent creation and versioned configuration	Tenant admin can create, test, publish and roll back agent configurations
FR-02	Document ingestion and lifecycle	Upload, parse, index, replace and delete supported document types
FR-03	Grounded Q&A; with citations	Answer using authorized sources or abstain/escalate
FR-04	Tenant-bound business tools	Execute allowlisted REST API operations with enforced permissions
FR-05	Conversation and escalation	Persist authorized sessions and produce support handoff summaries
FR-06	Observability and evaluation	Trace retrieval/tool path; report accuracy, latency and cost
FR-07	Deployment portability	Run in local Docker and defined cloud topology

3. Logical architecture



Figure 1. Trust-boundary-oriented logical architecture; arrows represent controlled interfaces.

The API creates trusted identity context. The policy service authorizes resource and tool operations. LangGraph coordinates business logic but cannot bypass retrieval permissions, secrets management or API authorization.

Core services

Service	Responsibilities
API / identity gateway	Authentication, request validation, tenant context, quotas, rate limiting
Agent orchestration	Intent routing, state, clarification, retrieval and tool approval
Knowledge engine	Ingestion, hybrid retrieval, reranking, citations and lifecycle management
Tool gateway	Schema validation, connector registry, secrets, permissions, audit and idempotency
LLM gateway	Provider routing, content policy, token budgets, redaction, usage accounting
Quality services	Traces, feedback, evaluations, release gates, cost and latency reporting

4. Retrieval-augmented generation



Figure 2. Secure document ingestion and knowledge retrieval lifecycle.

Document processing and indexing

- Supported initial sources: PDF, DOCX, Markdown, TXT, URL/HTML and curated structured records; add OCR only where needed.
- Scan files, enforce size/type limits, parse with page and section coordinates, normalize, chunk with metadata and generate embeddings.
- Store originals in encrypted object storage. Store metadata, versions and embeddings in PostgreSQL/pgvector.
- Use keyword + semantic retrieval, tenant and ACL filtering before ranking, optional reranking, deduplication and citation verification.
- Support upsert, reindex, soft-fail retries, deletion propagation and document versioning.

Answer contract

Every knowledge answer returns: answer text, source references (document, page/section, version), confidence/evidence flags, retrieval trace ID and escalation status. If no sufficient authorized evidence exists, the assistant clarifies, abstains or escalates; it must not fabricate citations.

5. Agent workflows and tool execution

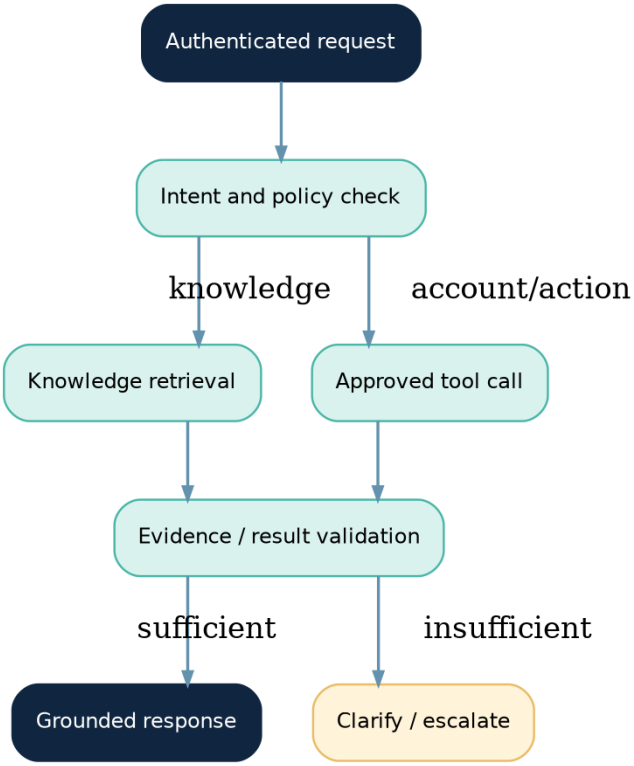


Figure 3. Controlled agent workflow: retrieval, tool execution and human escalation.

Tool registry specification

Field	Expected meaning
tool_id / version	Stable plugin identity and interface version
input_schema / output_schema	Typed JSON schemas; reject unrecognized or invalid parameters
permissions / policy	Tenant, role, resource and action authorization
auth_binding	Credentials resolved server-side via secrets manager
timeout / retry	Bounded network operations; safe retry and circuit breaker
risk_level / confirmation	Read-only by default; explicit approval for sensitive changes
audit / idempotency	Correlated execution log and replay protection

Plugin types: declarative allowlisted REST adapters for simple APIs; SDK-based connectors for proprietary authentication or complex workflows; human approval nodes for high-impact actions. LLM-generated API URLs, SQL statements or arbitrary shell commands are not trusted executable instructions.

6. Multi-tenant data and security architecture

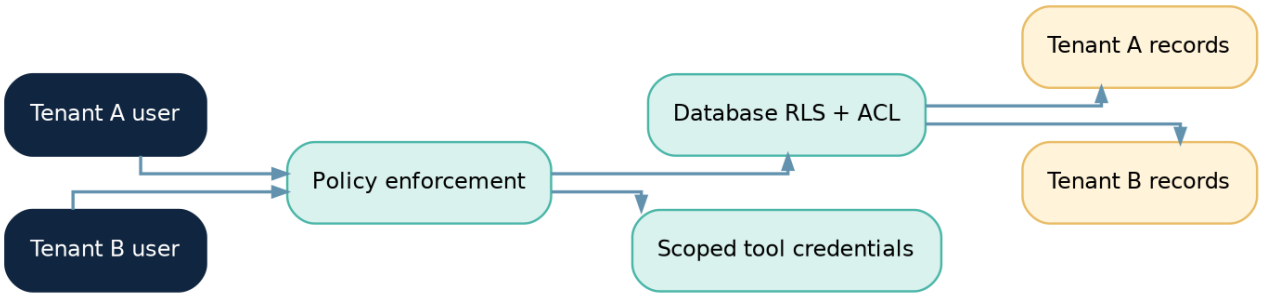


Figure 4. Logical isolation: only policy-authorized tenant data may cross into retrieval or tool context.

Security requirements - defense in depth

Boundary	Required control
Identity	OIDC/OAuth2, short-lived sessions, organization membership, MFA for administrators
Database	Tenant IDs + RLS; FORCE RLS where appropriate; limited service roles; transaction-scoped tenant context
Vectors / search	ACL applied before retrieval/reranking; no cross-tenant citation, cache or metadata leakage
Files / object storage	Tenant-scoped object policies; signed URL validation; encryption and malware scanning
Tools / APIs	Per-tenant connector credentials; least privilege; approval for write actions; SSRF defenses
LLM / prompts	Retrieved text treated as untrusted; data minimization; prompt-injection testing
Operations	Secrets management, centralized audit, security alerting, CI scans, periodic pen tests

Critical invariant: The LLM never independently chooses tenant identity, authorizes document access, retrieves secrets or grants tool privileges. The application establishes and enforces those decisions before protected data reaches the model.

Security acceptance tests

- Cross-tenant document, embedding, citation, cache and tool attempts fail in the predefined automated test suite.
- No known critical vulnerabilities remain open at production release; documented exceptions require explicit risk acceptance.
- Role and resource checks apply consistently to search, downloads, conversations, asynchronous jobs and admin functions.
- Prompt-injection and adversarial test cases include malicious document instructions, tool abuse and forged tenant identifiers.

7. Privacy, retention and data governance

Data type	Handling rule
Customer source documents	Customer-owned; versioned, tenant isolated and removable
Embeddings / extracts	Treated as customer-derived data; deleted/reindexed with original content
Conversations	Tenant-scoped; configurable retention and restricted export
Tool results	Minimized to necessary fields; redact PII in logs
Provider prompts/responses	Sent only to approved provider under verified retention/DPA controls
Backups / traces	Encrypted, access-limited, disclosed retention and eventual expiry

Cloud LLM API providers may have different retention, abuse-monitoring, regional processing and zero-retention eligibility rules. Provider selection is a documented customer policy, and no zero-retention promise is made until all providers and system components are verified. Customer data is not used to train ThinkHive models without explicit authorization.

Data lifecycle

Implement an account-wide deletion orchestration process covering files, extracts, embeddings, search indexes, cached results, conversations and job artifacts. Account for backup expiry, regulatory/legal holds and downstream subprocessor obligations. Expose export and deletion request tracking to tenant administrators.

8. Deployment and commercial packaging

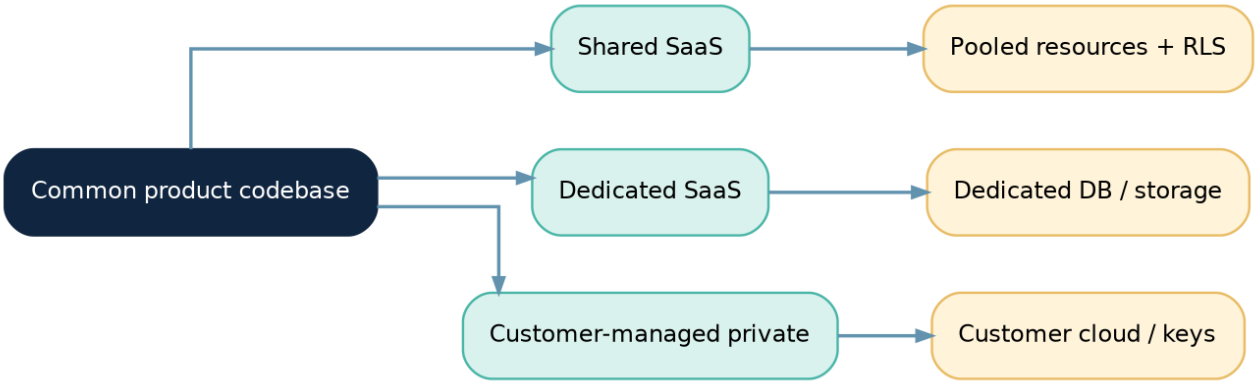


Figure 5. One product codebase with three packaging and infrastructure options.

Offering	Environment	Operating model
Cloud Starter / Business	Shared app and pooled tenant-isolated data	ThinkHive-hosted subscription + metered usage
Enterprise Dedicated	Dedicated tenant DB/storage and optionally app resources	Setup + contracted monthly/annual service
Private / Customer Managed	Customer account, cloud or on-premise as supported	License + implementation + annual maintenance

Cloud tenancy and data residency must be agreed per customer. Private deployment does not eliminate platform vulnerabilities or customer responsibilities; contracts must allocate infrastructure, patching, identity, monitoring, backup and incident response ownership.

9. API and configuration contracts

Indicative REST endpoints

Endpoint	Purpose
POST /v1/agents	Create agent configuration
POST /v1/agents/{id}/publish	Publish validated agent version
POST /v1/documents	Upload or register a source document
GET /v1/documents/{id}/status	Read ingestion/index status
POST /v1/conversations	Open authorized conversation
POST /v1/conversations/{id}/messages	Ask a question or submit a turn
POST /v1/tools/{tool_id}/invoke	Execute policy-controlled tool operation
POST /v1/escalations	Escalate unresolved conversation
GET /v1/evaluations/reports	Retrieve authorized quality scorecards

Configuration contract - representative fields

Agent: id, tenant_id, name, version, prompt_policy, model_policy, retrieval_profile, enabled_tool_ids, escalation_rules, channels, retention_profile, published_state. Tools: tool_id, schema, credential_reference, permissions, allowlisted host/path, rate_limit, confirmation_policy. Knowledge: collection_id, version, scope, ACL, chunking_profile, embedding_model, indexing_state. No raw secrets in configuration files or model context.

10. Data model outline

Entity	Key fields / relationships
Tenant	tenant_id, plan, region, status, retention_policy
Identity / Membership	user_id, tenant_id, role, group_ids
Agent / AgentVersion	agent_id, tenant_id, version, configuration, published_at
Document / Chunk	document_id, tenant_id, version, ACL; chunk_id, embedding
Conversation / Message	conversation_id, tenant_id, user_id; message_id, provenance
Connector / ToolExecution	connector_id, tenant_id, credential_ref; execution_id, decision, result
Escalation	conversation_id, ticket_reference, assignee, status
Evaluation / AuditEvent	dataset_id, metric, trace_id; actor, decision, timestamp

11. Quality assurance and SLO candidates

Measure	Proposed initial target	Method
Retrieval Recall@5	90%+ on representative test set	Gold-labeled retrieval evaluation
Citation support	95%+ on sampled grounded answers	Human and automated review
Cross-tenant isolation	Zero passes in adversarial test suite	Negative integration/security tests
Tool authorization	Zero unauthorized actions in suite	Policy and fuzz tests
Latency	Set P95 after measured pilot baseline	Production-like load test
Cost per resolution	Track and constrain per plan	Usage tracing and accounting
Escalation quality	Measure precision and resolution handoffs	Human review of real pilot transcripts

Targets are design-stage acceptance hypotheses, not achieved performance. Quantitative commitments must be based on test evidence, actual data and customer SLA terms. Online evaluation should include drift, high-risk intents, incorrect refusals, privacy, false citations and provider outages.

12. Delivery roadmap and staffing assumptions

Phase	Duration estimate	Acceptance deliverables
Foundation	Weeks 1-2	Service skeleton, identity, tenant context, schema, Docker
Knowledge engine	Weeks 3-5	Ingestion, hybrid search, versioning, citations
Agents + plugins	Weeks 6-8	LangGraph, tool gateway, API connectors, escalation
Productization	Weeks 9-11	Agent Studio lite, widget, evaluation, security, docs

Planning assumption: approximately one experienced full-time engineer with part-time frontend, QA and security support. Customer-specific connectors, regional compliance needs, SSO, high availability and private deployment may require additional effort.

Prioritized MVP backlog

Priority	Epic	Must-have outcome
P0	Identity + multi-tenancy	No cross-tenant access in API, storage, job and retrieval paths
P0	Knowledge ingestion + search	Document lifecycle, grounded answers, verified citations
P0	LangGraph + tools	Registered tools with server-side permissions and escalation
P1	Configuration + widget	Admin can onboard an agent without code changes
P1	Evaluation + tracing	Repeatable dataset-based release checks
P2	Dedicated/private delivery	Reproducible infrastructure variants and enterprise packaging

13. Demo and customer acceptance scenario

Create two isolated test organizations with realistic synthetic documents, subscriptions and ticketing APIs. Demonstrate an authorized documentation question with source citations; an account-specific lookup; a product recommendation; a confirmed support ticket; an uncertain query escalated to a human; and an adversarial attempt to fetch the other tenant's data, which must be denied. Reconfigure the same platform as a second support template to demonstrate reuse.

14. Product ownership and commercialization

Strategy: retain reusable ThinkHiveLabs platform IP where contracts permit; license the platform to customers while treating customer-owned data and separately assigned deliverables according to the signed agreement. Review Upwork/client work-for-hire provisions before presenting existing platform components as retained IP. Customer pricing should distinguish implementation, subscriptions/license, support and model or infrastructure usage.

Model	Monetization	Launch rationale
Implementation-first	Fixed-scope delivery + integration fees	Early cash flow and product feedback
Shared SaaS	Subscription + usage allowance/overages	Repeatable SME deployments and recurring revenue
Dedicated enterprise	Setup + recurring hosting/operations	Isolation and customer-specific operational terms
Private license	License + implementation + maintenance	Customers requiring infrastructure control

Illustrative commercial pricing discussed internally is not a market-validated quotation and should not be included in customer offers until cost, support and competitive research is completed.

15. Key risks, decisions and exit criteria

Risk / decision	Mitigation / gate
Cross-tenant data disclosure	Independent policy controls, full negative-test coverage, pen test before release
Hallucinated / weakly supported output	Evidence thresholds, citations, evaluation, abstention and escalation
LLM provider privacy / cost	Provider gateway, per-tenant policies, budgeting and retention review
Connector abuse / SSRF	Approved hosts and operations, typed schemas, restricted egress
Operational overload	Modular monolith, queues, managed services and clear incident ownership
Scope creep	Deliver fixed initial workflows and defer visual builder/advanced autonomy
Customer IP conflict	Written license and deliverable ownership clauses before engagement

Release gate for first external pilot

- End-to-end demo proves two-tenant isolation, source-grounded answers, tool execution and escalation.
- Architecture threat model and critical controls reviewed by an independent security practitioner.
- Automated tests, data deletion path, backups and documented incident-response procedures exist.
- Provider privacy, subprocessor and retention terms confirmed for the selected pilot geography.
- Pilot statement of work defines scope, acceptance criteria, limitations, service ownership and intellectual property.

16. Reference documentation

References are supporting technical reading; implementations must be validated against current vendor documentation, laws and contract terms.

- PostgreSQL Row Security Policies - https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- pgvector - https://github.com/pgvector/pgvector
- LangGraph Documentation - https://docs.langchain.com/oss/python/langgraph/overview
- OWASP Top 10 for LLM Applications - https://genai.owasp.org/llm-top-10/
- OpenAI API Data Controls - https://developers.openai.com/api/docs/guides/your-data
- AWS SaaS Tenant Isolation - https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/tenant-isolation.html

Final architecture decision: Build one secure, reusable product; deliver value first through customer implementations; evolve into managed SaaS while preserving enterprise dedicated/private deployment options. Authorization, tenant isolation, data lifecycle and the LLM/tool gateways are mandatory platform primitives.